

SQL QUERY NAME	DESCRIPTION	PROVES
anon_permissions (TEMPORARY DEVELOPMENT ACCESS – WILL BE REMOVED ONCE THE APP IS OFFICIALLY USING AUTHETICAED USERS)	temporarily gives the anon role permission to read the ingredients table and creates an RLS policy that allows anonymous requests to view all ingredient rows	Enabled the earlier App Deploy proof of concept to read ingredient data from Supabase before the real user authentication ws connected.
01_test_inventory_batch_join	tests that inventory_batches is correctly linked to ingredient through ingredient_id	A physical inventory batch can successfully retrieve the descriptive information for the ingredient it belongs to. For example, the Fresh Basil batch correctly connects to the Fresh Basil ingredient record.
02_test_inventory_status_calculation	tests the core inventory calculations Iron Chef will use for the dashboard: usable quantity and cost at risk	Iron Chef can take raw batch records and turn them into meaningful operational information, such as: Fresh Basil has 5 bunches usable and \$12.50 of inventory at risk.
03_create_current_inventory_status_view	creates a reusable SQL view named current_inventory_status so Iron Chef can query calculated inventory information without having to rebuild the same logic every time	This creates a permanent database object that acts like a virtual inventory summary table for the application.
04_test_current_inventory_status_view	tests the current_inventory_status view I created in query 03	The app can eventually query one clean database view instead of recreating inventory calculations in frontend code

05_secure_restaurant_memberships_read	secures the restaurant_memberships table so authenticated user can only read their own membership records	A user cannot query the membership table and see every restaurant/user relationship in the application, they can only see the memberships that belong to their own account.
06_secure_inventory_batches_read	secures the inventory_batches table so authenticated user can only read inventory batches belonging to restaurants they are actually members of	A user from Black Fox Café can read only their own inventory, but not inventory belonging to another restaurant.
07_secure_ingredients_authenticated_read	gives authenticated users permission to read ingredients, then limits them to ingredients belonging to restaurants where they have membership	Builds restaurant level security for ingredient data. A Black Fox Café user can see Black Fox Café ingredients, but ingredients from another restaurant are hidden by RLS. (row level security)
^^queries 05 – 07 grant access and create the security rules on the real tables^^		
08_test_authenticated_rls	simulates Samantha as a logged in authenticated user and attempts to read restaurant_memberships, ingredients, and inventory_batches while RLS is active	Confirms the positive RLS test: an authenticated Black Fox Café member is successfully allowed to access inventory data belonging to their own restaurant. The returned Fresh Basil batch confirms the membership based security policy is permitting legitimate access
09_test_cross_restaurant_rls_block	temporarily creates a second restaurant with its own ingredient and inventory batch, then simulates Samantha as an authenticated Black Fox Café user and	Confirms the negative side of RLS, even though another restaurants records exist in the database, a user who does not have a membership for that restaurant cannot see its ingredients or inventory batches. The query returning 0 rows

	attempts to read the other restaurants data	proves cross restaurant data isolation is working.
10_secure_current_inventory_status_view_read	gives authenticated users permission to read current_inventory_status view, which provides the calculated inventory summary used by the app	Makes the inventory summary view available to logged in users while still relying on the underlying RLS protected tables to control which restaurant data they can see
^^query 10 grants access to the view created in 03, which then relies on the security rules already protecting its underlying tables^^		
11_test_authenticated_inventory_status_view	simulates Samantha as an authenticated user and queries the current_inventory_status view while the underlying RLS policies are active	Confirms that the secured inventory summary view works correctly for a legitimate logged in user. Samantha can see Black Fox Café's calculated inventory data, including items with 0 usable quantity.
^^query 11 verifies that the view from 03 (now granted to authenticated users by 10, actually works under Samantha's RLS restrictions^^		
12_test_cross_restaurant_inventory_view_block	temporarily creates inventory for a second restaurant, simulates Samantha as an authenticated Black Fox Café user, and attempts to access the other restaurants data through current_inventory_status view	Confirms that the inventory summary view does not bypass RLS. Even though the view combines and calculates data from multiple tables, Samantha still cannot see inventory belonging to a

		restaruant where she has no membership. Returning 0 rows proves the view preserves cross restaurant data isolation.
13_diagnose_auth_user_state	inspects the Iron Chef Supabase Auth user to confirm its authentication state without modifying the account	The test user is email confirmed, has password stored, and uses the email provider. This rules out an unconfirmed account or social login only account as the cause of the login failure.
14_remove_temporary_anon_ingredient_access	removes the temporary anonymous ingredient read access used by the early App Deploy proof of concept	Iron Chef no longer needs unauthenticated database access, ingredient data is now accessed through the authenticated application flow and protected RLS policies
15_secure_ingredients_authenticated_insert	allows authenticated users to insert ingredients only when the new row's restaurant_id matches a restaurant they belong to	this starts the Create side of CRUD security, a Black Fox Café member should be able to add a Black Fox Café ingredient, but not insert one into another restaurant
16_test_authenticated_ingredient_insert	simulates the real authenticated Iron Chef manager account creating an ingredient for Black Fox Cafe	an authenticated Black Fox Café member can successfully add an ingredient to their own restaurant while RLS is active
17_test_cross_restaurant_ingredient_insert_block	tests that RLS blocks an authenticated user from adding an ingredient to a restaurant they are not a member of	a user cannot add ingredients to another restaurants inventory

18_secure_inventory_batches_authenticated_insert	allows authenticated restaurant members to create inventory batches only for their own restaurant and only for ingredients belonging to that same restaurant	inventory batches cannot be created for another restaurant or attached to an ingredient from the wrong restaurantdo
19_test_authenticated_inventory_batch_insert	tests that an authenticated Black Fox Café member can create a valid opening inventory batch for an ingredient belonging to their restaurant	an authenticated restaurant member can securely add inventory quantity, expiration, and cost for their own restaurant
20_test_cross_restaurant_inventory_batch_insert_block	tests that an authenticated user cannot create an inventory batch for a restaurant they do not belong to	a user cannot add inventory data to another restaurant
21_test_mismatched_ingredient_batch_insert_block	tests that an inventory batch cannot be created using an ingredient that belongs to a different restaurant	inventory batches cannot be linked to another restaurants ingredients
22_check_expired_inventory_quantity	checks whether chicken breast has any real inventory batches in Supabase (the app seemed to have automatically removed expired inventory)	chicken breast exists as an ingredient, but currently has no inventory batch or physical quantity recorded in the live database
23_secure_inventory_quantity_manager_update	allows authenticated managers to update inventory quantities only for their own restaurant	managers can change their restaurant's inventory quantity while RLS prevents unauthorized restaurant updates

^^got an error saying that this policy already exists (see 23a)^^

23a_inspect_existing_inventory_update_security	inspects the existing manager inventory update RLS policy	confirms whether managers can update quantities only for inventory belonging to their own restaurant
24_test_manager_inventory_discard_update	tests that a Black Fox Café manager can approve a discard by updating an expired batch's remaining quantity to zero	a manager can remove expired inventory from their own restaurant after explicit approval
25_test_cross_restaurant_inventory_update_block	tests that a manager cannot update inventory belonging to another restaurant	cross restaurant inventory updates are blocked and the other restaurants quantity remains unchanged